

BUSN 41210: Financial Analytics

Lecture 7: Clustering

Dacheng Xiu

Chicago Booth

September 21, 2026

© 2026 Dacheng Xiu, Chicago Booth Restricted Materials. All Rights Reserved.

Outline

- ▶ Supervised versus unsupervised data analysis
- ▶ Model-based clustering: **mixture models**
- ▶ **K-means** algorithm
- ▶ Spectral clustering
- ▶ Hierarchical clustering

Supervision

We've focused on models for $E(y|\mathbf{x})$. Today is about models for $\mathbf{x}$.
The goal in everything we do has been *Dimension Reduction*.

DR: move from high dimensional $\mathbf{x}$ to low-D summaries.

Dimension reduction can be supervised or unsupervised.

Supervised: Regression and Classification

HD $\mathbf{x}$ is projected through β into 1D $\hat{y}$

Outside info (y) supervises how you simplify $\mathbf{x}$.

Unsupervised: Mixture and Factor Models

$\mathbf{x}$ is modeled as built from a small number of components.

You're finding the simplest representation of $\mathbf{x}$ alone.

We always want the same things: low deviance, without overfit.

What is Clustering? And Why?

Clustering: dividing up data into groups (clusters), so that points inside each group are more “similar” to each other than to points outside the group.

Why cluster? Two main uses

- ▶ **Summary:** (DR) deriving a reduced representation of data
- ▶ **Discovery:** looking for new insights into the data structure. Clustering can also help with predictions. However,

clustering should not be confused with classification!

- ▶ **In classification**, we have data for which the groups are **known** and we try to learn what differentiates them to assign future labels
- ▶ **In clustering**, we have data for which the group labels are **unknown** and try to learn the groups themselves as well as what differentiates them.

Clustering: Unsupervised Dimension Reduction

Group observations into similar 'clusters', and understand the rules behind this clustering.

- ▶ **Demographic Clusters:** Soccer moms, NASCAR dads.
- ▶ **Consumption Clusters:** Jazz listeners, classic rock fans.

Collaborative Filtering

Group individuals into clusters, and model average behavior for each.

- ▶ **Industry Clusters:** Competitor groups, supply chains.

Clustering is largely an **exploratory** technique.

- ▶ **model-based** methods (mixture models)
- ▶ **nonparametric** methods (spectral clustering)
- ▶ **"heuristic"** methods (hierarchical clustering)

Sometimes, it is useful to have clusters organized in a hierarchy.

Some Examples of Common Clustering Applications

- ▶ **Marketing:** Discover distinct groups in their customer bases, and then use this knowledge to develop targeted marketing programs.
- ▶ **Land use:** Identification of areas of similar land use in an earth observation database.
- ▶ **Insurance:** Identifying groups of motor insurance policy holders with a high average claim cost.
- ▶ **Urban planning:** Identifying groups of houses according to their house type, value, and geographical location.
- ▶ **Earthquake studies:** Observed earthquake epicenters should be clustered along continent faults.
- ▶ **Finance:** Identify groups of stocks with similar characteristics to create portfolios that try to profit from differences within/across groups.

Clustering Traders

28

Y. Aït-Sahalia and C. Brunetti / *Journal of Econometrics* 217 (2020) 20–45

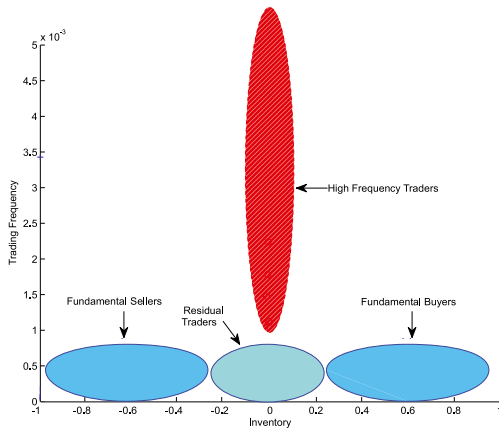


Fig. 4. Clustering analysis: trading frequency and inventory held by different types of traders, August 2010. Notes: This figure plots the clusters of traders identified using the two criteria of trading frequency and carried-over inventory. LFTs are located in the blue shaded areas, with a distinction between fundamental buyers and fundamental sellers, while the red shaded area marks the cluster of HFTs, with no individual accounts indicated to preserve their anonymity. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

What is Good Clustering?

- ▶ A good clustering method produces high quality clusters consisting of:
 - ▶ High intra-class similarity
 - ▶ Low inter-class similarity
- ▶ The quality of a clustering result depends on both the similarity measure used by the method and on its implementation.
- ▶ The quality of a clustering method is also measured by its ability to discover potential hidden patterns.

The K -means Mixture Model

The fundamental model of clustering is a **mixture**:

observations are random draws from K populations, each with different average characteristics.

Suppose you have K possible means for each observed $\mathbf{x}_i$:

$$\mathbb{E}[\mathbf{x}_i \mid \mathbf{k}_i] = \boldsymbol{\mu}_{\mathbf{k}_i}, \text{ where } \mathbf{k}_i \in \{1 \dots K\}$$

e.g., if $\mathbf{k}_i = 1$, $\mathbf{x}_i$ is from cluster 1: $\mathbb{E}[x_{i1}] = \mu_{11} \dots \mathbb{E}[x_{ip}] = \mu_{1p}$.

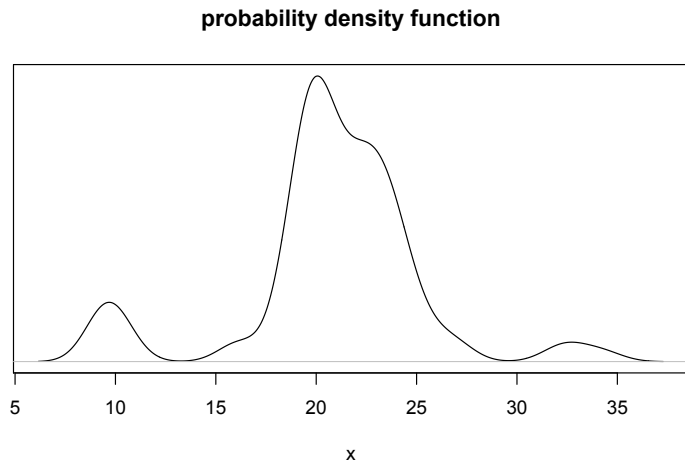
Each mean μ_j has an associated probability or “weight” in the mixture.

For new $\mathbf{x}$ with unknown k ,

$$\mathbb{E}[\mathbf{x}] = p(k=1)\boldsymbol{\mu}_1 + \dots + p(k=K)\boldsymbol{\mu}_K$$

DR: Given $\boldsymbol{\mu}_k$'s, you discuss data in terms of K different types, rather than trying to imagine all possible values for each $\mathbf{x}$.

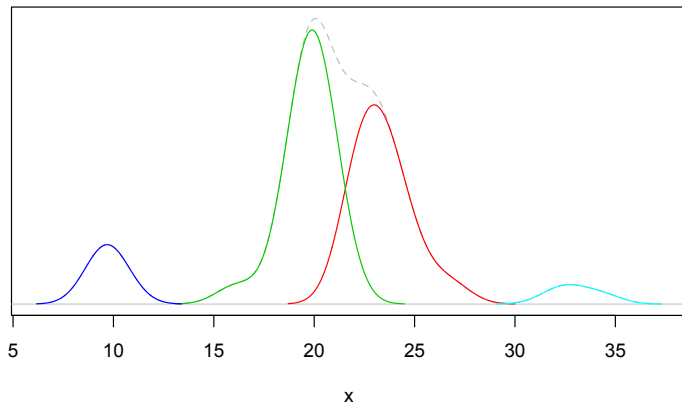
Mixture Model: without knowing membership ' k '



The *marginal* density has multiple modes; one for each μ_k .

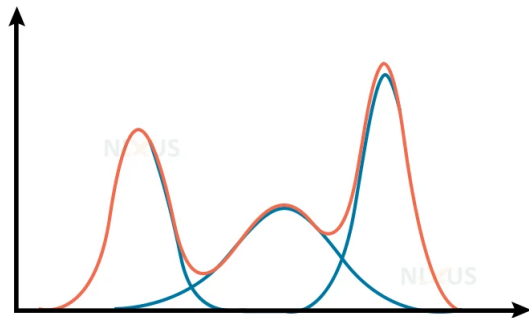
Mixture Model: breaking into K components

probability density function



Here, we have $K = 4$ different cluster centers. Should it be 5?

Gaussian Mixture Model



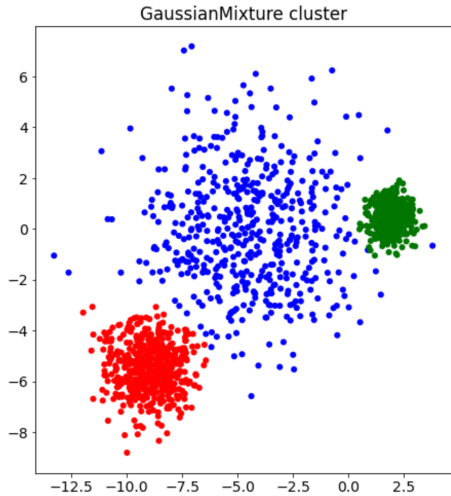
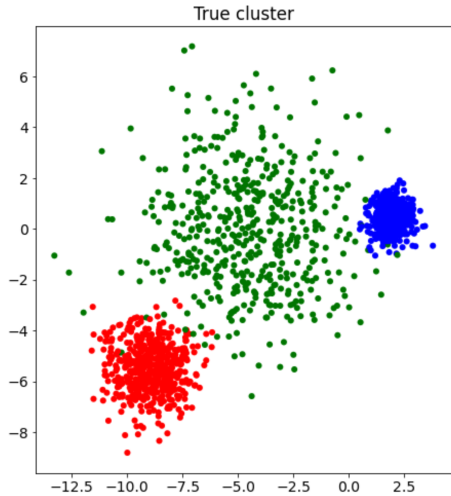
- ▶ Assuming each data point is generated with a certain probability π_j from K Gaussian distributions with $\mathcal{N}(\mu_j, \Sigma_j)$, $j = 1, 2, \dots, K$.
- ▶ Likelihood is not unimodal and may stuck at local minimums.

The `GaussianMixture` object implements the expectation-maximization (EM) algorithm for fitting mixture-of-Gaussian models.

```
from sklearn.mixture import GaussianMixture
gmm = GaussianMixture(n_components=3)
gmm.fit(X)
gmm_clusters = gmm.predict(X)
```

The output of `GaussianMixture` includes

- ▶ “weights_”: The weights of each mixture components.
- ▶ “means_”: The mean of each mixture component.
- ▶ “covariances_”: The covariance of each mixture component.



The K -means Method

- ▶ The K -means algorithm is one of the oldest but still very popular clustering method due to its simplicity
- ▶ It applies to situations where all variables are of the continuous type (or are transformed to)
- ▶ The square of the Euclidean distance is used as a dissimilarity measure between pairs of observations (i, j) :

$$d_{i,j} = \|\mathbf{x}_i - \mathbf{x}_j\|^2,$$

where $\|\cdot\|$ is the Euclidean distance.

The K -means Criteria (Objective Function)

- ▶ Let $K \geq 1$ be the number of clusters (chosen a priori). For a given assignment $\mathbf{k}$, the within-cluster variation is

$$W(\mathbf{k}) = \sum_{k=1}^K \sum_{\mathbf{k}_i=k} \sum_{\mathbf{k}_j=k} \|\mathbf{x}_i - \mathbf{x}_j\|^2$$

- ▶ which is proportional to

$$\sum_{k=1}^K N_k \sum_{\mathbf{k}_i=k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2,$$

where N_k is the number of points in the k -th cluster and $\boldsymbol{\mu}_k$ its associated mean called centroid.

K-means: Chicken-Egg Problem

For given K , the goal is to find clusters so that the within-cluster variability is small.
We do not know cluster memberships k_i and we do not know centroids (K-means) μ_k



Chicken-and-Egg Problem

But!

1. If we knew k_i : we can easily estimate μ_k
2. If we knew μ_k : we can easily estimate k_i

Solution: iterate between 1. and 2.

K-means Algorithm

The K -means algorithm looks for an assignment $\mathbf{k}$ that minimizes $W(\mathbf{k})$.

Chicken: If you know membership $\mathbf{k}_i$ for each $\mathbf{x}_i$, then estimate

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{i:\mathbf{k}_i=k} \mathbf{x}_i$$

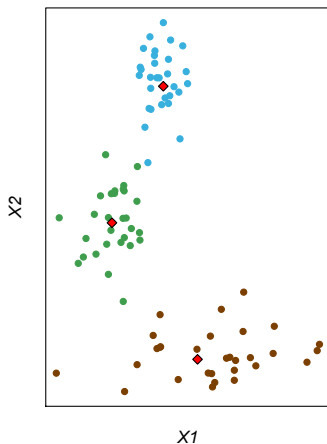
where $\{i : \mathbf{k}_i = k\}$ are the n_k observations in group k .

Egg: If you know means μ_k , find $\mathbf{k} = k_1 \dots k_n$ to minimize the sum-of-squares

$$\sum_{k=1} \sum_{i:\mathbf{k}_i=k} (\mathbf{x}_i - \hat{\mu}_k)^2$$

Mixture deviance: total sums of squares **within** each cluster.

Give K -means the x_i 's, and it gives you back the k_i 's



In words: K-means

- ▶ Label each point based on the closest centroid (mean)
- ▶ Replace each centroid by the average of the points in the cluster

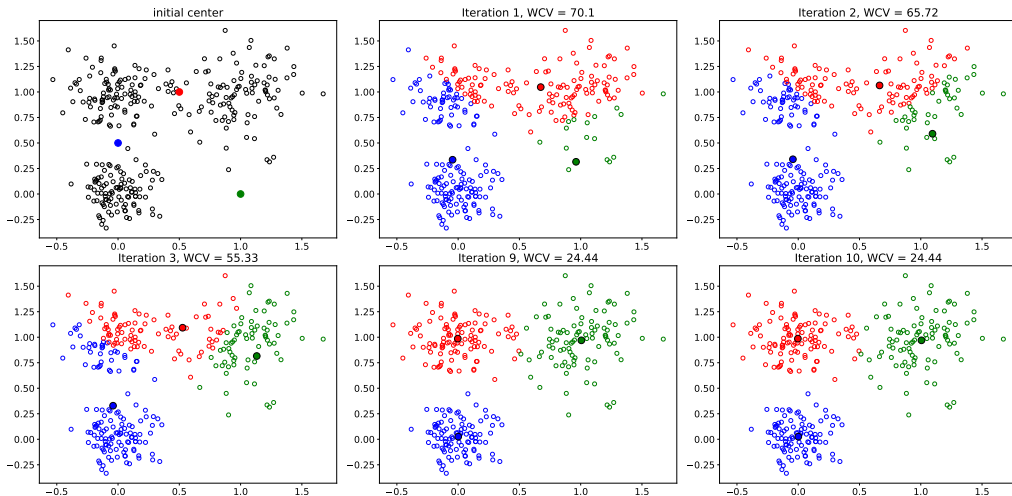
The algorithm starts at random μ_k , and changes k_i 's until the sum of squares stops improving.

Solution depends on start location.
Try multiple, take the best answer.

Choosing K : For most applications, everything is descriptive.
So try a few and use clusters that make sense to you.

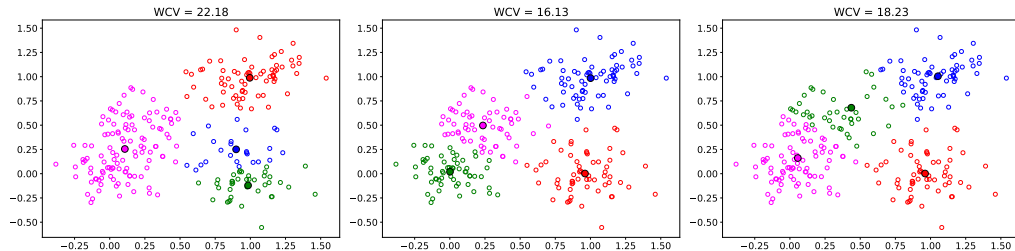
K-means Example

Here $X_i = (X_{i1}, X_{i2})$, $n = 300$, and $K = 3$



K-means Example, Multiple Runs

Here $X_i = (X_{i1}, X_{i2})'$, $n = 250$, and $K = 4$, the points are not as well-separated



These are results of running the K-means algorithm with different initial centers (chosen randomly over the range of the X_i 's). We choose the second collection of centers because it yields the **smallest within-cluster variation** (mixture deviance)

Clusters **X** (numeric!) into **n_clusters** groups.

```
from sklearn.cluster import KMeans
kmeans=KMeans(n_clusters=3,init='random',n_init=1,max_iter=10)
kmeans.fit(X)
```

Output each cluster's center coordinates:

```
np.around(kmeans.cluster_centers_,2)
array([[ 0.   ,  0.03],
       [-0.   ,  0.99],
       [ 1.01,  0.97]])
```

Information about cluster labels:

```
from collections import Counter
Counter(kmeans.labels_)
Counter(0: 101, 1: 105, 2: 94)
```

Within cluster sum of squares:

```
round(kmeans.inertia_,2)
24.44
```

Scaling for K -means

The algorithm minimizes total [squared] distance from center, *summed across all dimensions of $\mathbf{x}$* .

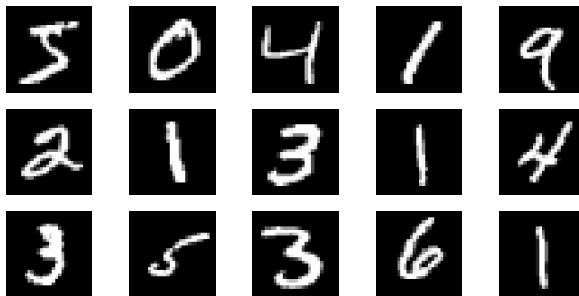
Scale matters: if you replace x_j with $2x_j$, that dimension counts twice as much in determining distance from center
(and will have more influence on cluster membership).

Standard solution is to standardize:

$$\text{cluster on scaled } \tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{\text{sd}(x_j)}$$

The MNIST Database of Handwritten Digits

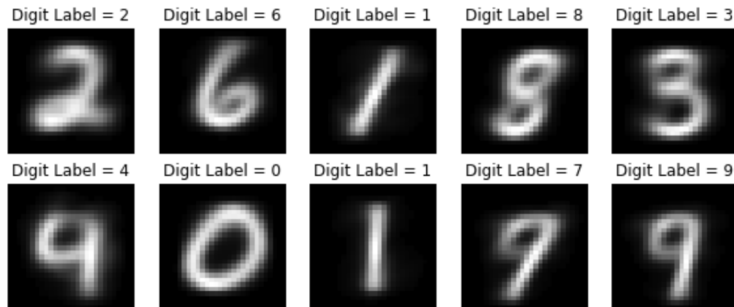
- ▶ The MNIST database of handwritten digits, available from Yann LeCun's [Website](#), has a training set of 60,000 examples, and a test set of 10,000 examples.
- ▶ The digits have been size-normalized and centered in a fixed-size image.
- ▶ The training data: $60,000 \times 784$ matrix; the testing data: $10,000 \times 784$ matrix.



10-means Clustering on MNIST

The most natural choice of K is 10. Why?

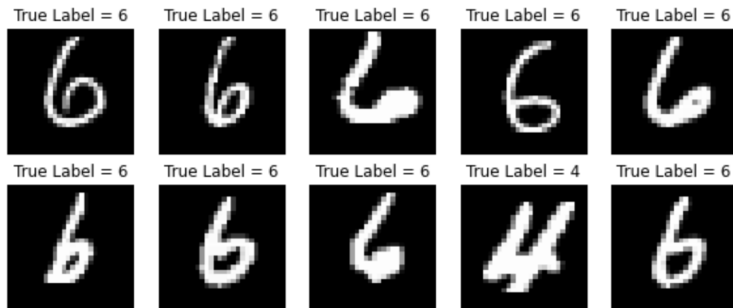
Display the center of each cluster:



If using the majority label in the clusters as the predicted label, the model has the accuracy of : 57.84 % on training data and 59.45 % on testing data.

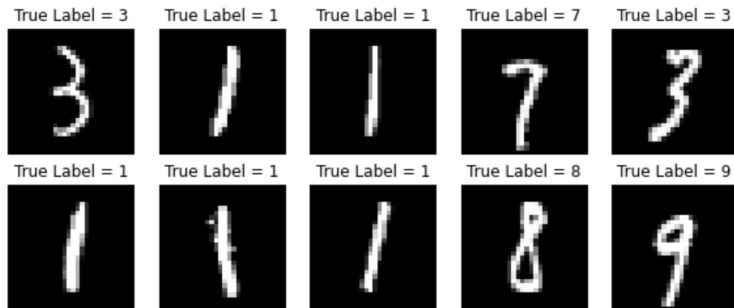
10-means Clustering on MNIST

10 random images of the cluster whose majority label is 6



10-means Clustering on MNIST

10 random images of the cluster whose majority label is 1



OUR ANALYSIS SHOWS THAT THERE ARE
THREE KINDS OF PEOPLE IN THE WORLD:
THOSE WHO USE K-MEANS CLUSTERING
WITH $K=3$, AND TWO OTHER TYPES WHOSE
QUALITATIVE INTERPRETATION IS UNCLEAR.



Choosing K

1st order advice: most often clustering is an exploration exercise, so choose the K that makes the most sense to you.

But, we can apply data-based model building here:

1. Enumerate models for $k_1 < k_2 < \dots < k_K$.
2. Use a selection tool to choose the best model for new $\mathbf{x}$.

Step one is easy. Step two is a tougher.

For example, for CV you'd want to have high OOS $p_{k_i}(\mathbf{x}_i)$.

But you don't know k_i ! This is a *latent* variable.

There's no ground truth like y to compare against.

AIC and BIC for K-means

We *can* use IC to select K .

- ▶ Deviance is (as always) $D = -2\log LHD$, which for K -means is the total sum of squares (slide 10).
- ▶ df is the number of μ_{kj} : $K \times p$. (where p is dimension of $\mathbf{x}$)

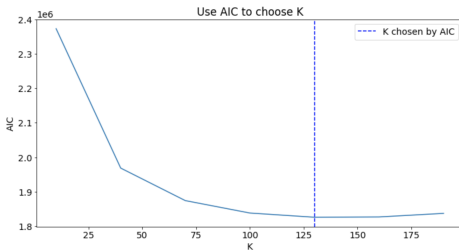
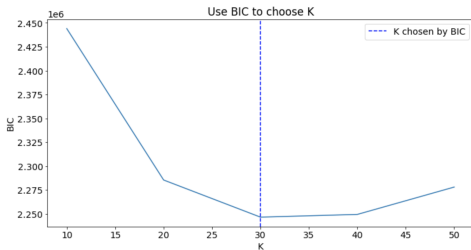
Then our usual AIC and BIC formulas apply.

Beware: the assumptions behind both AIC and BIC calculations are only roughly true for K -means.

These tools are lower quality here than in regression.

You're often better off just using descriptive intuition.

BIC and AIC for MNIST K -means



BIC likes $K = 30$, AIC likes 130.

Both are way more complicated than is useful.

With $K = 30$, the model has the accuracy of : 75.085% on training data, and 76.29% on testing data.

Re-visiting the K-means Model

In minimizing sums-of-squares, K -means targets the model

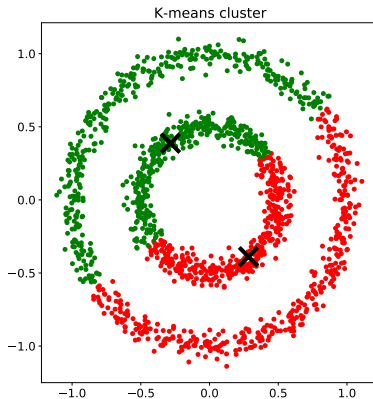
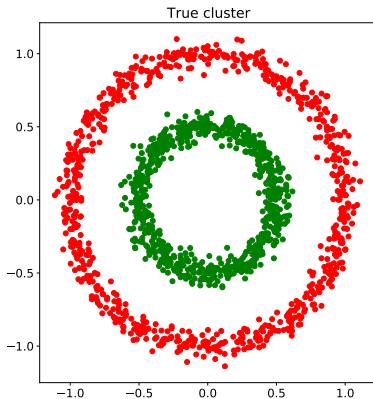
$$p_k(\mathbf{x}) = \prod_j N(x_j \mid \mu_{kj}, \sigma^2)$$

$\Rightarrow$ independence across dimensions (no multicollinearity) and uniform variance (same σ for all j , which is why scale matters).

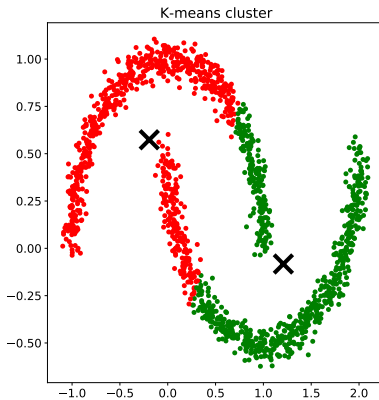
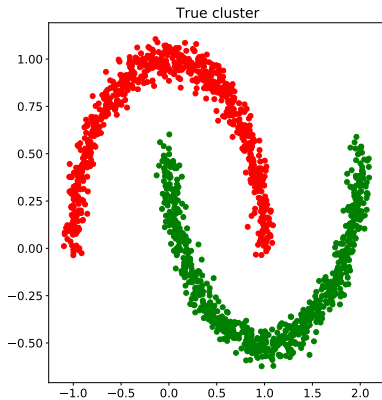
Despite being a silly model, this tends to do a decent job of clustering when $\mathbf{x}$ consists of *continuous* variables.

It does a worse job for $\mathbf{x}$ made up of dummies or counts.

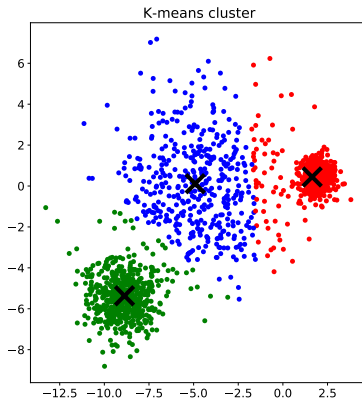
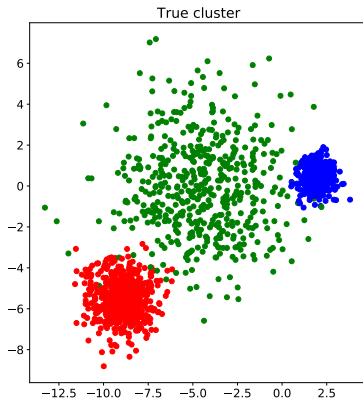
When K-means Fail: Non-Gaussian



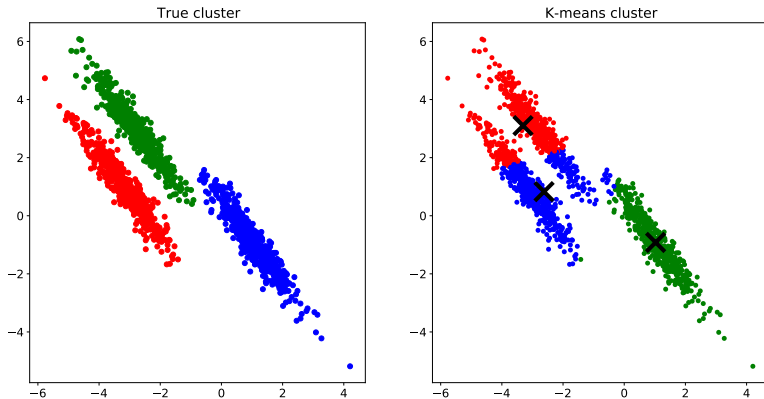
When K-means Fail: Non-Gaussian



When K-means Fail: Unequal Variances



When K-means Fail: Anisotropic Data



Why K -means Fails on These Shapes

K -means assigns each point to the nearest **center**: it uses straight-line distance to a single point.

On the two rings, that is the wrong notion of “close”. Two points on the *same* outer ring are on average 1.28 apart; a point on the outer ring and a point on the inner ring are on average 1.05 apart. Measured this way, the rings are not clusters at all.

But every point on the outer ring is close to its **neighbors**, and through them it is linked to every other point on that ring. **Similarity is local**: a cluster is a set of points connected by chains of nearby points.

So build the graph of who is near whom, and cluster the graph instead of the coordinates.

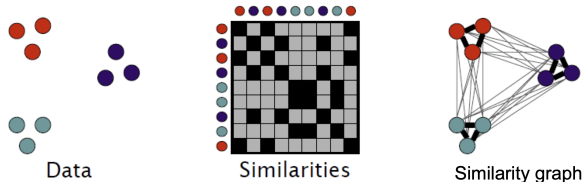
The Similarity Graph

Similarity Graph: $G(V,E,W)$

V – Vertices (Data points)

E – Edge if similarity > 0

W - Edge weights (similarities)



Each data point is a vertex. Connect i and j if they are similar, with weight $W_{i,j} \geq 0$; W is symmetric.

K-nearest-neighbor graph: $W_{i,j} = 1$ if i is among the K nearest neighbors of j or j is among those of i ; otherwise 0. **Fully connected graph:** $W_{i,j} = \exp(-d^2(i,j)/c)$, $d(i,j)$ a distance.

Two rings, 10-nearest-neighbor graph: **no edge between the rings**. In the graph, the rings are two separate pieces.

The Graph Laplacian

Let G be diagonal with $G_{i,i} = \sum_j W_{i,j}$, the total weight at vertex i , and define

$$L = G - W.$$

A six-point graph made of two triangles, with no edge between them:

$$W = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}, \quad L = \begin{pmatrix} 2 & -1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ -1 & -1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & -1 & -1 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & -1 & 2 \end{pmatrix}.$$

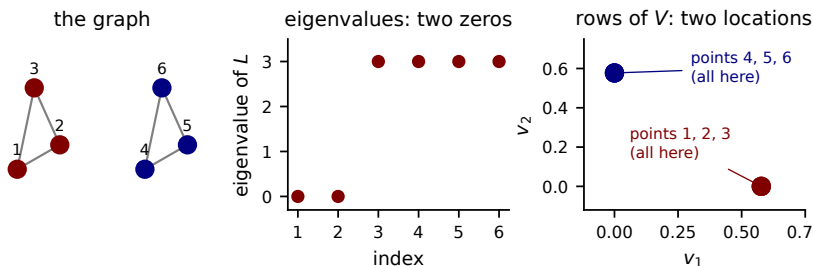
Two facts, each one line:

- ▶ Every row of L sums to zero, so $L\mathbf{1} = 0$: the constant vector has eigenvalue 0.
- ▶ $v^\top L v = \frac{1}{2} \sum_{i,j} W_{i,j} (v_i - v_j)^2 \geq 0$ for every v , so every eigenvalue of L is ≥ 0 .

Zero Eigenvalues Count the Pieces

$v^\top L v = 0$ exactly when $v_i = v_j$ for every edge $W_{i,j} > 0$, that is, when v is **constant on each connected piece** of the graph.

So if the graph has K separate pieces, L has exactly K eigenvectors with eigenvalue 0: the indicators of the pieces (or any rotation of them).



Six-point graph: eigenvalues $0, 0, 3, 3, 3, 3$; the two eigenvectors with eigenvalue 0 are $(1, 1, 1, 0, 0, 0)/\sqrt{3}$ and $(0, 0, 0, 1, 1, 1)/\sqrt{3}$. As columns of V : **row i of V is the same for every point in a piece**. Six points become two locations.

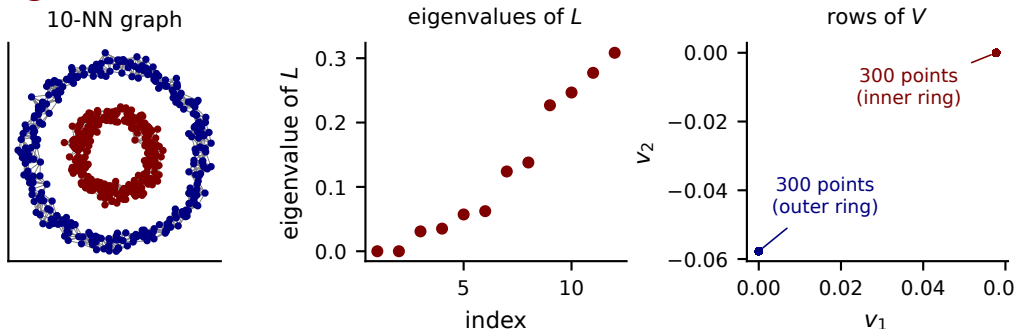
Spectral Clustering Algorithm

1. Build the similarity matrix W (K -nearest neighbors, or a Gaussian kernel).
2. Form $L = G - W$.
3. Compute the K eigenvectors of L with the *smallest* eigenvalues, $V_{n \times K} = [v_1, \dots, v_K]$.
4. Represent point i by row i of V : the K numbers $(v_{1,i}, \dots, v_{K,i})$.
5. Run K -means on these n new points.

Why K -means comes back. If the graph has exactly K pieces, the rows of V take K distinct values and K -means has nothing to do. Real clusters are only *nearly* disconnected: the K smallest eigenvalues are small rather than zero, the rows of V are nearly constant on each cluster, and K -means finishes the job.

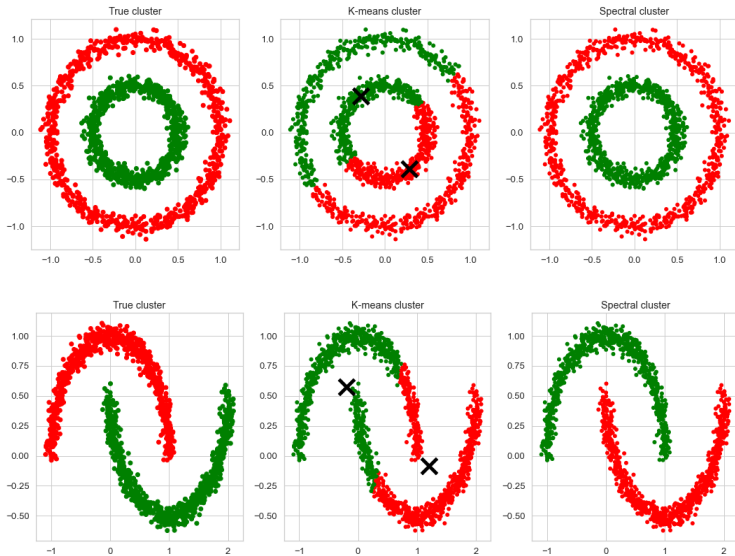
In Python: `SpectralClustering(n_clusters=K, affinity='nearest_neighbors')`. It uses a normalized version of L (rows divided by the $G_{i,i}$); the idea is the same.

Two Rings, Worked

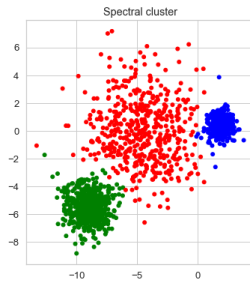
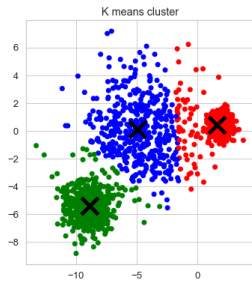
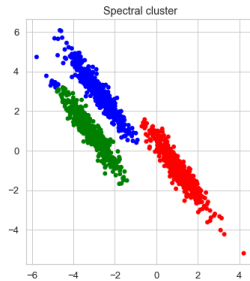
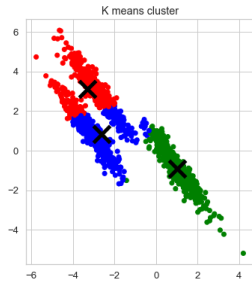


- ▶ 600 points, 10-nearest-neighbor graph: no edge crosses between the rings, so the graph has two pieces.
- ▶ Eigenvalues of L : 0, 0, 0.031, 0.036, ... — two zeros, then a gap. **The number of eigenvalues near zero tells you K .**
- ▶ The rows of V sit at two locations. K -means on them recovers the rings exactly; K -means on the original coordinates gets 51% right.

Spectral Clustering



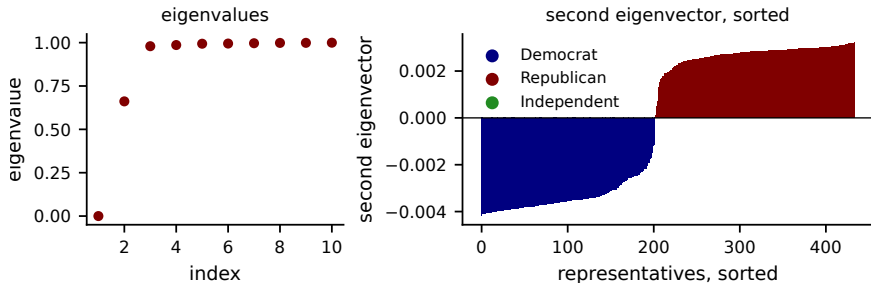
Spectral Clustering



Example: Clustering Legislators Based on Votes

- ▶ Data for U.S. House of Representatives Roll Call Votes. Each line corresponds to a single member of the House, and it contains the results for that member on 669 roll calls: 1 for Yea, -1 for Nay, 0 for no vote.
- ▶ We have 434 Representatives.
- ▶ Goal: We use voting behavior to reveal legislators' political view (liberal or conservative).

Clustering Results



- ▶ Fully connected graph, $W_{i,j} = \exp(-d_{i,j}/669)$ with $d_{i,j}$ the number of roll calls on which i and j differ.
- ▶ The graph is connected, so exactly one eigenvalue is 0 — the constant vector, which says nothing. The eigenvalues are 0, 0.66, 0.98, 0.98, ...: one small one after the zero, then a plateau. **Two groups.**
- ▶ The **second eigenvector** gives one coordinate per member. Its sign matches the party of all 433 Democrats and Republicans.

Clustering Results

The members at the two ends of the second eigenvector:

most negative:

Schakowsky, Woolsey, Lee, Baldwin, G. Miller — all Democrats

most positive:

Blackburn, Foxx, Akin, McHenry, Neugebauer — all Republicans

The coordinate orders members from one end of the House to the other, not just into two groups.

	Name	second_eig	Group
355	Schakowsky	-0.065527	D
429	Woolsey	-0.064166	D
229	Lee	-0.064044	D
11	Baldwin	-0.063970	D
275	Miller, George	-0.063947	D
...	...	...	...
28	Blackburn	0.049626	R
136	Foxx	0.049632	R
3	Akin	0.049699	R
256	McHenry	0.050037	R
288	Neugebauer	0.050160	R

434 rows x 3 columns

From K-means to Hierarchical Clustering

Recall two properties of **K-means** clustering:

- ▶ It fits exactly K clusters (as specified)
- ▶ Final clustering assignment depends on the chosen initial cluster centers

Hierarchical clustering is an alternative that does not rely on any underlying model

- ▶ Hierarchical clustering produces a sequence of nested cluster memberships.
- ▶ No need to choose initial starting positions and the number of clusters.
- ▶ Data points that are similar will end up in the same cluster.

There are different ways to measure similarity.

At one end, all points are in their own cluster, at the other end, all points are in one cluster

Agglomerative vs Divisive

Two types of hierarchical clustering algorithms

Agglomerative (i.e., bottom-up):

- ▶ Start with all points in their own group
- ▶ Until there is only one cluster, repeatedly: merge the two groups that have the smallest dissimilarity

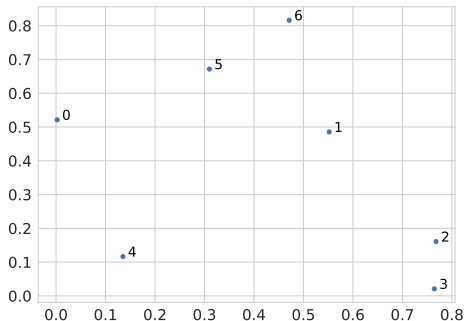
Divisive (i.e., top-down):

- ▶ Start with all points in one cluster
- ▶ Until all points are in their own cluster, repeatedly: split the group into two resulting in the biggest dissimilarity

Agglomerative strategies are **simpler**, we'll focus on them.

Simple Example

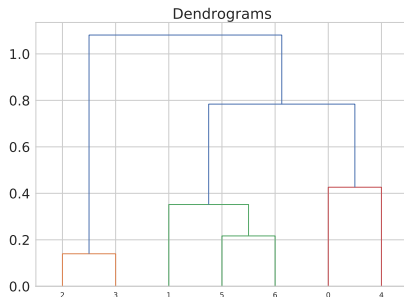
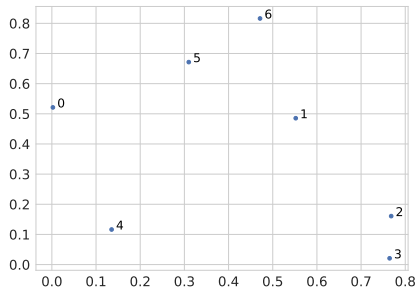
Given these data points, an agglomerative algorithm might decide on a clustering sequence as follows:



- ▶ Step 1: $\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}$;
- ▶ Step 2: $\{0\}, \{1\}, \{2, 3\}, \{4\}, \{5\}, \{6\}$;
- ▶ Step 3: $\{0\}, \{1\}, \{2, 3\}, \{4\}, \{5, 6\}$;
- ▶ Step 4: $\{0\}, \{1, 5, 6\}, \{2, 3\}, \{4\}$;
- ▶ Step 5: $\{0, 4\}, \{1, 5, 6\}, \{2, 3\}$;
- ▶ Step 6: $\{0, 4, 1, 5, 6\}, \{2, 3\}$;
- ▶ Step 7: $\{0, 1, 2, 3, 4, 5, 6\}$.

Dendrogram

We can also represent the sequence of clustering assignments as a **dendrogram**:



Note that **cutting the dendrogram** horizontally partitions the data points into clusters

What's A Dendrogram?

Dendrogram: convenient graphic to display a hierarchical sequence of clustering assignments. This is simply a tree where:

- ▶ Each node represents a group
- ▶ Each leaf node is a singleton (i.e., a group containing a single data point)
- ▶ Root node is the group containing the whole data set
- ▶ Each internal node has two children nodes, representing the the groups that were merged to form it

Remember: the choice of similarity measure determines how we merge groups of points

Hierarchical Clustering of WSJ News

We have data from 180 topics, each being a vector of length 18,433.



See full graph on [this website](#).

Source: Bybee, Kelly, Manela, and Xiu, Business News and Business Cycles (2024, Journal of Finance).